Northwind

The Dawn Manifest

1.29°N 103.85°E · SINGAPORE · A VENDING NETWORK AT SCALE

The problem. A new engineer, fluent in Python but new to Spark, has built the nightly manifest job for a vending network across Singapore. It runs, and every answer is correct. But it is written without knowing what the engine does underneath, so it does far more work than it needs to. The lead reviews it part by part: how Spark defers work, how it plans, how it reads less, how it lays data out, and how it moves less across the cluster. Same answers, a fraction of the cost.

FIVE PARTS AND GROUNDWORK

I Execution model
II Optimiser and engine
III Read less
IV Data layout
V Shuffle less

THE SETTING · SINGAPORE

Correct code, and far more costly than it needs to be.

Daniel Tan leads data engineering for Northwind Automated Retail, a network of vending machines across Singapore: MRT concourses, office lobbies, schools, hospitals, malls. Most machines sell a handful of items a day. One, on a busy interchange, sells a tenth of everything on its own.

Each night a batch job turns the day's vends into the **Restock Manifest**: which machines to refill, and with what, before the route trucks set out. **Wei Chen**, new to the team and new to Spark, has just rewritten that job.

Wei's code is correct. Every query returns the right answer, and it passed Wei's own testing. But Wei is new to Databricks and does not yet know what the engine does beneath a line of PySpark, so the job does far more work than it needs to.

Daniel sits down to review it, the way he would any pull request. Reading it line by line, one question runs through the whole case: **where is Wei paying for work the job never needed to do?**

HOW DANIEL REVIEWS THE JOB

Groundwork. What a Parquet file is, what a Delta table is, and how to read the plan `explain()` prints.

Part I · Execution model. Why some steps are free and some are not.

Part II · Optimiser and engine. What rewrites the query, and what runs it.

Part III · Read less. Where the job reads more than it needs to.

Part IV · Data layout. How the table is organised, and why that decides the cost.

Part V · Shuffle less. Where data moves across the cluster, and the one machine that makes it move most.

THE WORKED EXAMPLE

One machine on an interchange sells roughly twenty-four times the median. It turns up in nearly every part of the review: it is why grouping by machine behaves as it does, and why Wei's job spends unevenly. Follow it through, and the cost of the code explains itself.

GROUNDWORK · WHAT THE TABLE IS

Parquet and Delta

Before reviewing Wei's queries, Daniel checks what the job writes. Not one file but many, and inside each the data sits down the columns, not across the rows. Delta keeps a log so a write is all-or-nothing. Half the savings Daniel will find rest on this format alone.

MECHANISM

CSV stores a record at a time, so reading one field reads them all. `Parquet` stores each column separately with statistics per block. `Delta` adds a log; a failed write leaves no trace.

ROW FORMAT · CSV



every byte read

COLUMNAR · PARQUET



two columns read, four skipped

GROUNDWORK · READING THE PLAN

explain(), logical and physical

Daniel cannot review what he cannot see. Every query carries two plans: the logical one, what Wei asked for, and the physical one, how Spark will run it. Reading that plan, printed by `explain()`, is how Daniel judges each of Wei's cells.

MECHANISM

Read a plan bottom to top: the scan reads, Project stays put, an Exchange is a shuffle. The DAG is not in `explain()`; it is in the profile under `See performance` once an action runs.



PART I · EXECUTION MODEL

Lazy evaluation

Wei worried a long chain of steps was slow to build. Daniel shows why it is not: filter, add a column, select, and the cell returns at once. Two million rows, untouched. The cost has not arrived because the work has not run. This cell was never the problem.

MECHANISM

Transformations extend a plan and return immediately, touching no data. The work runs only when an **action**, count, show, write, asks for a result. `count()` is where the cost lands.

TRANSFORMATIONS · LAZY



ACTION · RUNS ALL

PART I · EXECUTION MODEL

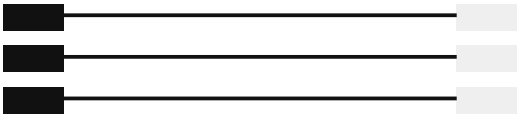
Narrow versus wide

Here is the first real finding. Wei filters to one machine in an instant, but grouping vends by machine makes the whole cluster churn. The difference is not size, it is whether rows must move. That single distinction explains most of what Wei's job spends.

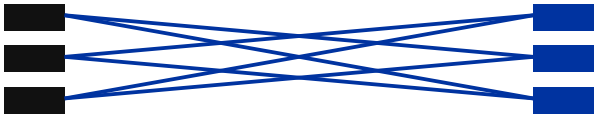
MECHANISM

A **narrow** transformation (`filter`, `select`) needs only the rows in hand. A **wide** one (`groupBy`, `orderBy`, `distinct`) needs rows from every partition, so Spark inserts a shuffle.

NARROW



WIDE · SHUFFLE



PART II · OPTIMISER AND ENGINE

Catalyst rewrites the question

Wei put a filter in what looked like the wrong place, after the group-by, and feared it was slow. Daniel checks the plan: the filter has moved down to the scan. Catalyst rewrote it. The engine does not run Wei's code literally, so this one needs no change.

MECHANISM

Catalyst is the optimiser. It turns the logical plan into an efficient physical one: pushing filters to the scan, choosing join strategy, deciding where to shuffle. Everything `explain()` prints is its output.



PART II · OPTIMISER AND ENGINE

Photon, and the Tungsten fallback

Reviewing Wei's join, Daniel sees a merge hint. It strips the `Photon` prefix from the operators and hands the step to the slower JVM engine. Part of Wei's cost is simply which engine runs each step, and this hint pushed one off the fast one.

MECHANISM

Photon is the C++ vectorised engine; its nodes are prefixed `Photon`. When it cannot run a node it falls back to the JVM engine, whose memory and codegen are **Tungsten**. Unprefixed names mean it fell back.



PART III · READ LESS

Predicate pushdown

Now Daniel hunts for waste in earnest. Where Wei filtered by day, the plan shows the test on the scan itself, so Parquet skips whole blocks. Good instinct from Wei, even if by luck: here the job already reads less than Daniel feared.

MECHANISM

A predicate on a scanned column becomes a `RequiredDataFilters` entry on the scan, and Parquet skips blocks whose statistics cannot match. On Free Edition the line is not called `PushedFilters`; that is the open-source name.

WITHOUT

read all | filter

WITH PUSHDOWN

read only matching rows

RequiredDataFilters: [vend_date]

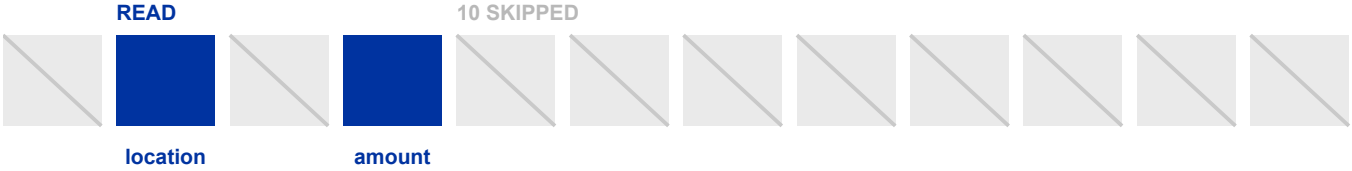
PART III · READ LESS

Column pruning

The vend row is fourteen columns wide and the manifest needs two, but Wei wrote select("*"). Daniel narrows it to the two columns and the scan reads a fraction of the row. Real cost recovered, from one obvious line in the review.

MECHANISM

Selecting a subset narrows the `PhotonScan` output and `ReadSchema` to just those columns; the rest are never touched. CSV cannot do this, it reads every byte of every row.



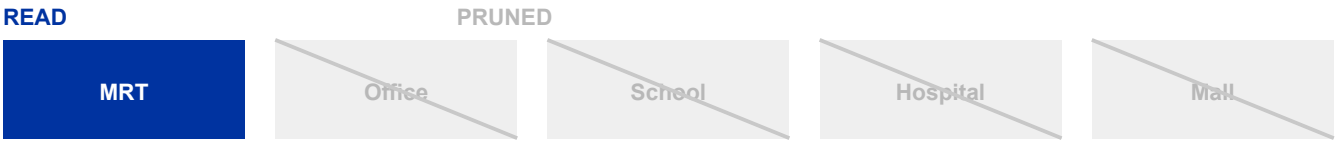
PART III · READ LESS

Partition pruning

The biggest waste in Wei's job is here. The manifest is built one day at a time, yet Wei's table is unpartitioned, so a one-day query still scans the whole quarter. Partitioned by date, the same filter opens one folder of ninety.

MECHANISM

`partitionBy` splits the table into one folder per value. A filter on the partition column becomes a `PartitionFilters` entry and Spark eliminates whole folders before reading. It works only on the partition column.



WHERE THE COST WAS HIDING

This is most of the slowdown Daniel came to explain. Reading a full quarter to build one day's manifest is ninety times the work needed. Partitioning by `vend_date` is the difference between scanning two million rows and scanning twenty thousand.

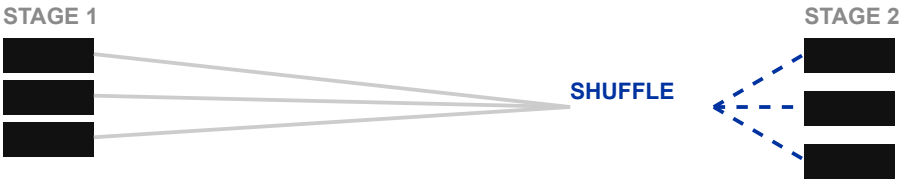
PART IV · DATA LAYOUT

Small files and OPTIMIZE

Wei wrote the table in sixty-four files, thinking more files meant more parallelism. Past a point it is only more opens and more tasks. Daniel runs `OPTIMIZE` to pack them back into a few large ones, and the file count falls.

MECHANISM

Too many small files means one open per file, so overhead swamps the work. `OPTIMIZE` bin-packs them into fewer large files. `DESCRIBE DETAIL` shows `numFiles` before and after.



PART IV · DATA LAYOUT

Partitioning versus liquid clustering

Partitioning by date fixed one query but the manifest also filters by machine and by location. Partitioning by all three would shatter the table into tiny folders. Daniel reaches for clustering, which gets the same skipping without the folder explosion.

MECHANISM

`partitionBy` splits into folders: one fixed column, rewrite to change it. `CLUSTER BY` keeps one set of files and skips by file statistics on up to four keys you can change later. Databricks now recommends clustering under 100 TB.

COALESCE	SWITCH JOIN	SPLIT SKEW
auto → real size	to broadcast	giant machine

PART V · SHUFFLE LESS

Broadcast join

The last cost is movement. To name each machine's location Wei joined vends to the machines dimension and let it run. The plan shows a broadcast anyway: machines is tiny, so Spark copies it to every task and joins in place. One place Wei got for free.

MECHANISM

When one side is small, Spark copies it to every executor and joins in place, a **broadcast hash join**, no shuffle of the big side. `broadcast(machines)` forces it, regardless of the size threshold.



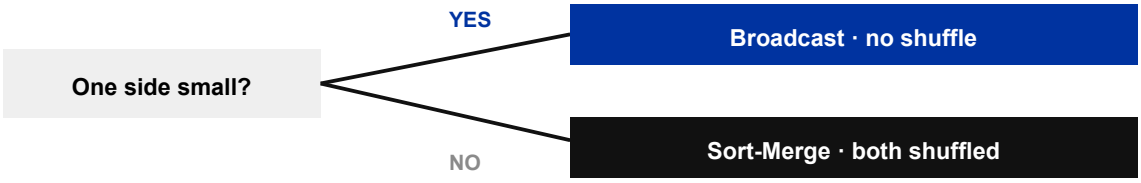
PART V · SHUFFLE LESS

Sort-merge join

Elsewhere Wei added a merge hint to the same kind of join. Now both sides shuffle and sort by key, and the join drops off Photon. Two costs at once, where the broadcast paid neither. Daniel flags it: this is the expensive shape to avoid.

MECHANISM

A **sort-merge join** shuffles and sorts both sides by the join key, then merges. It is the general strategy when neither side is broadcastable. `.hint("merge")` forces it, and also forces the Photon fallback.



PART V · SHUFFLE LESS

Adaptive query execution

And here is the interchange machine again, the one that sells a tenth of everything. In Wei's group-by its task runs long after the rest, one bar past all the others. Daniel checks the profile: AQE already split that partition, so this needs understanding, not a fix.

MECHANISM

AQE re-plans mid-flight once it sees real partition sizes: coalescing tiny shuffle partitions and splitting skewed ones. `explain()` shows `isFinalPlan=false`; the final plan and DAG are in the profile. Always on, cannot be switched off on serverless.

COALESCE
auto → real size

SWITCH JOIN
to broadcast

SPLIT SKEW
giant machine

THE MACHINE THAT EXPLAINS THE SHAPE

The interchange machine sells roughly twenty-four times the median, so its group is the straggler that decides when the whole stage finishes. This one machine, threaded through every part, is why the job's cost was uneven. Understand it and the shape of the job stops being a mystery.

CAPSTONE · THE REVIEW, COMPLETE

Wei's query, and the reviewed one

Daniel puts the whole manifest query side by side: Wei's version, and the reviewed one. Wei's uses a sort-merge join and filters after the join, so it reads the whole quarter. The reviewed version is the same answer built on every finding: a clustered table, the filter first, the dimension broadcast. Same result, and Daniel can point to exactly where every unit of Wei's cost went.

WEI'S VERSION

Sort-merge, so both sides shuffle and Photon falls back. The filter sits above the join, so every row is read before most are discarded. Correct, but costly in three places at once.

REVIEWED

Clustered Delta, so files skip by machine and date. Filter first, so the scan reads one month. Broadcast the dimension, so the join moves nothing and stays on Photon.

THE FIVE PLACES THE COST LIVED

1 · The file format. Columnar Delta over CSV unlocks pushdown, column pruning and skipping.

2 · The join. Broadcasting the small dimension removes a shuffle and keeps the join on Photon.

3 · The filter order. Filtering first reaches the scan as `RequiredDataFilters` and reads less.

4 · The layout. `CLUSTER BY (machine_id, vend_date)` so a single day skips most files.

5 · The skew. AQE splits the interchange machine's partition so it stops deciding the runtime.

YOU ARE DANIEL · PREPARATION

Questions before the seminar

Bring plans, not prose. For each, run it against `vends` and `machines` and be ready to show the physical plan, and to say where the cost lives.

01 Narrow or wide. Classify all twenty transformations below by whether the plan gains an Exchange. Argue the surprising ones.

02 Two dialects. Show the same join once on Photon and once forced to fall back. Name every operator that changes.

03 Pushdown. Filter `vends` to one day and point to the line on the scan. Why is it not called `PushedFilters`?

04 Layout. When would you partition `vends` by date, and when cluster it instead? Give the plan for each.

05 Join strategy. Force broadcast, then sort-merge, on `vends` join `machines`. What does the merge cost beyond the shuffle?

06 The giant. Group by machine and find the straggler. Explain what AQE does with it, and where you see it.

01 · THE TWENTY TO CLASSIFY

01 · `filter`	02 · `select`	03 · `groupBy`	04 · `orderBy`
05 · `withColumn`	06 · `distinct`	07 · `join`	08 · `broadcast(dim)`
09 · `coalesce`	10 · `repartition`	11 · `union`	12 · `dropDuplicates`
13 · `sort`	14 · `where`	15 · `sample`	16 · `cube`
17 · `rollup`	18 · `sortWithinPartitions`	19 · `explode`	20 · `count`

Correct code, now cheap as well.

Wei's manifest returned the right numbers from the start. After the review it returns them from a far cheaper plan, and Wei now knows why each change mattered: which steps are free and which move data, what Catalyst rewrites, which engine runs each node, where the job read more than it needed, and the one machine whose skew shapes the whole run. The next job Wei writes will start closer to right.

I Free steps and costly ones

II What plans, what runs

III Reading less

IV Laying data out

V Moving less, and the skew